

Capítulo 4 — El despliegue

Unidad de estudio · Edición ampliada con fundamentos teóricos

Capítulo 4 — El despliegue	1
4.1. Alcance de la clase	3
4.2. Por qué existe un proxy inverso: origen y diseño	3
4.3. Qué hace Easypanel	4
4.3.1. El flujo de despliegue	5
4.4. Organización de los repositorios	6
4.5. Creación del proyecto	7
4.5.1. Dominio predeterminado	8
4.6. Conexión con GitHub	8
4.7. Despliegue del backend	8
4.7.1. Source	8
4.7.2. Environment	9
4.7.3. Domains & Proxy	9
4.7.4. Despliegue y verificación	9
4.8. Anatomía del enrutamiento: cómo decide Traefik	10
4.8.1. El modelo: punto de entrada, enrutador y servicio	10
4.8.2. Alojamiento virtual en funcionamiento	11
4.8.3. Emisión del certificado: el protocolo ACME	11
4.9. Despliegue del frontend	12
4.9.1. Source	12
4.9.2. Environment	12
4.9.3. Domains & Proxy	13
4.9.4. Verificación del arranque	13
4.10. La política de mismo origen y el mecanismo CORS	13
4.10.1. Qué es un origen	13
4.10.2. Por qué existe la política de mismo origen	13
4.10.3. La verificación previa	14
4.10.4. Qué cuenta como origen distinto	15
4.10.5. Qué NO protege CORS	15
4.11. Verificación funcional	15
4.11.1. Pestaña Red	15
4.11.2. Pestaña Consola	16
4.11.3. Casos límite	16
4.12. Operación	16
4.13. Seguridad y evolución del transporte web	17

4.13.1. Cabeceras de seguridad.....	17
4.13.2. HTTP/2 y HTTP/3	17
4.14. Errores frecuentes	17
4.15. Actividades	18
4.16. Síntesis	19
4.17. Referencias y lecturas complementarias	19

4.1. Alcance de la clase

Esta clase reúne todo lo anterior. El dominio de la Clase 1, el servidor de la Clase 2 y las imágenes de la Clase 3 convergen en dos servicios publicados en internet con certificado válido.

El capítulo tiene una particularidad respecto de los anteriores: casi todo su procedimiento consiste en completar formularios, y sin embargo es donde más fallos aparecen. La razón es que cada campo de esos formularios configura una pieza de infraestructura que el panel no muestra —un enrutador, un certificado, una variable que se lee al arrancar— y un valor mal puesto produce un síntoma que aparece en otro lado. Un número equivocado en un formulario se manifiesta como un error del servidor; una cadena de texto mal escrita se manifiesta como un problema de red. Por eso este capítulo desarrolla primero **el modelo del proxy inverso y el de la política de mismo origen**, y recién después recorre los formularios: sin esos dos modelos, la tabla de diagnóstico de la sección 4.14 es una lista de recetas para memorizar, y con ellos es una consecuencia.

Lectura previa obligatoria: el archivo `README.md` del repositorio del proyecto, en particular la sección sobre CORS.

Al finalizar la clase, cada grupo debe tener la calculadora funcionando en `https://calculadora.sudominio` consumiendo la API en `https://api.sudominio`.

Contenidos

1. Origen y objetivos de diseño del proxy inverso y del alojamiento virtual.
2. Qué hace Easypanel por debajo y el flujo de construcción y publicación.
3. Organización del proyecto en dos repositorios.
4. Creación del proyecto y conexión con GitHub.
5. Despliegue del backend.
6. Anatomía del enrutamiento en Traefik: puntos de entrada, enrutadores y servicios.
7. El protocolo ACME y la emisión automática de certificados.
8. Despliegue del frontend.
9. La política de mismo origen y el mecanismo CORS.
10. Verificación funcional y operación.
11. Seguridad y evolución del transporte web.
12. Diagnóstico de errores frecuentes.

4.2. Por qué existe un proxy inverso: origen y diseño

En la Clase 1 apareció, casi de pasada, un dato que ahora conviene desplegar: **un único servidor con una única dirección IP puede atender muchos dominios distintos**. Esa capacidad no es obvia y no siempre existió; su historia explica la arquitectura completa de este capítulo.

La primera versión de HTTP que se usó masivamente, normada en 1996 como RFC 1945, tenía una carencia que hoy parece increíble: **la petición no decía a qué sitio iba dirigida**. El cliente abría una conexión a una dirección IP y pedía `GET /index.html`; el servidor no tenía forma de saber si el usuario había escrito `empresa-a.com` o `empresa-b.com`, porque esa información se había consumido en la resolución de nombres y no viajaba en la petición. La consecuencia era estructural: **cada sitio web del mundo necesitaba su propia dirección IP**. Con un espacio de 4.300 millones de direcciones y una web que crecía exponencialmente, ese modelo tenía fecha de vencimiento.

HTTP/1.1, publicado en 1997, resolvió el problema con una decisión mínima y enorme: hizo **obligatoria la cabecera Host**. A partir de ese momento la petición lleva escrito el nombre que el usuario tipeó, y un mismo proceso escuchando en una única dirección puede mirar ese valor y decidir a quién corresponde. La técnica se llama **alojamiento virtual** y es la que sostiene económicamente a la web tal como la conocemos.

La aparición del cifrado agregó un problema y su solución. Si la conexión se cifra antes de enviar la petición, el servidor tiene que elegir qué certificado presentar **antes** de poder leer la cabecera Host, que viaja adentro del canal cifrado. Durante años eso significó que el alojamiento virtual y HTTPS eran incompatibles. La extensión **SNI** (*Server Name Indication*), incorporada a TLS en 2003, resolvió el huevo y la gallina haciendo que el cliente declare el nombre solicitado **en claro**, al inicio del saludo, antes de que exista canal cifrado. Es el mismo mecanismo mencionado en la sección 1.3, y es la razón por la que hoy un VPS de diez dólares puede alojar veinte sitios con certificado propio.

Sobre esa base se consolidó una pieza de arquitectura: el **proxy inverso**. Un proxy común representa al cliente ante muchos servidores; un proxy inverso hace lo opuesto, representa a muchos servidores ante todos los clientes. Ocupa los puertos 80 y 443, termina el cifrado, lee el nombre solicitado y reparte hacia el proceso interno que corresponda. La ventaja no es solo el reparto: al concentrar la terminación TLS en un único lugar, los certificados se gestionan una vez y las aplicaciones internas quedan liberadas de saber nada sobre criptografía.

La última pieza es reciente y es la que hace posible este práctico. Los proxies inversos clásicos — nginx, Apache— se configuran con archivos de texto que hay que editar y recargar cada vez que aparece un servicio nuevo. Con contenedores que nacen y mueren permanentemente, ese modelo no escala. **Traefik**, aparecido en 2015, invirtió el planteo: en lugar de leer un archivo, **descubre la configuración observando el entorno**. Consulta al motor de Docker qué contenedores existen y qué etiquetas tienen, y arma sus reglas de enrutamiento sobre la marcha. Cuando Easypanel levanta un contenedor nuevo con la etiqueta correcta, nadie edita ni recarga nada: Traefik ya lo vio.

PARA ENTENDER

Quedate con la idea, porque es la que ordena todo el capítulo: **hay un solo proceso atendiendo el 80 y el 443 de tu servidor, y no es tu aplicación**. Es Traefik. Tu API y tu frontend están adentro, escuchando en puertos que desde afuera no existen. Todo lo que configures en "Domains & Proxy" no le está hablando a tu aplicación: le está hablando al portero.

4.3. Qué hace Easypanel

Easypanel es una interfaz de administración sobre tres componentes que ya existían. Entender qué hace cada uno permite diagnosticar cuando algo falla.

Componente	Función	Qué pasa si falla
Docker	Construye las imágenes y ejecuta los contenedores	La construcción no termina
Traefik	Proxy inverso: recibe todo el tráfico de los puertos 80 y 443 y lo distribuye	502, 404 o el dominio no responde
Let's Encrypt	Emite los certificados TLS automáticamente	El sitio queda sin HTTPS

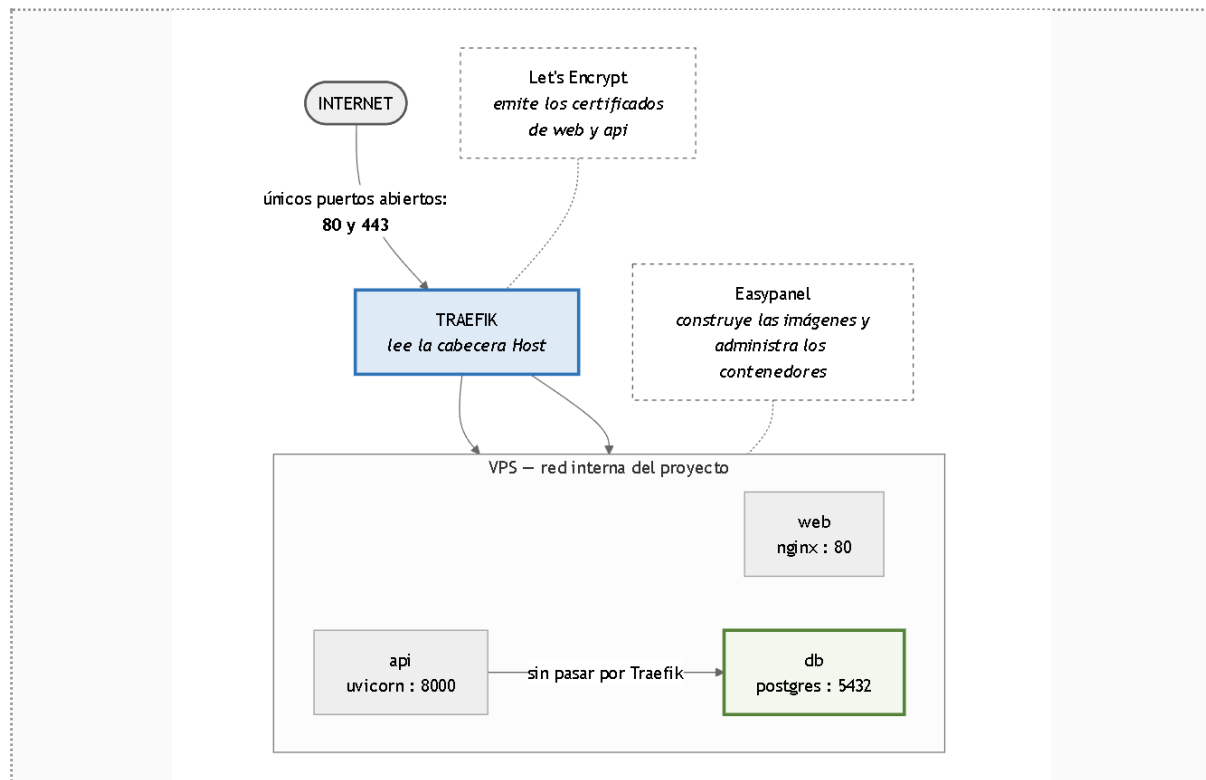


Figura 4.1. Arquitectura interna. Traefik es el único punto de entrada; los servicios se comunican entre sí por la red interna del proyecto.

Lo que Easypanel aporta por encima de esos tres no es funcionalidad sino **una capa de traducción**: convierte lo que se completa en un formulario en etiquetas de Docker, variables de entorno y órdenes de construcción. Es exactamente la categoría de producto que se llama *plataforma como servicio*, y su valor es reducir el conocimiento necesario para operar; su costo es que, cuando falla, el error aparece expresado en el vocabulario del componente subyacente y no en el del formulario que se completó.

💡 PARA ENTENDER

Easypanel no inventa nada. Es una cara linda para tres herramientas que podrías configurar a mano, y que en un trabajo real vas a terminar configurando a mano alguna vez. Lo importante no es aprender dónde están los botones —eso cambia con cada versión— sino entender **qué componente hace qué**. Cuando algo falle, esa es la pregunta que te da la respuesta: ¿esto es problema de Docker, de Traefik o del certificado?

4.3.1. El flujo de despliegue

Easypanel no recibe archivos cargados manualmente. El flujo es siempre el mismo:

1. El desarrollador publica el código en un repositorio remoto.
2. Easypanel clona ese repositorio en el servidor.
3. Si encuentra un `Dockerfile` en la ruta indicada, construye la imagen con él.
4. Ejecuta un contenedor a partir de la imagen resultante.
5. Traefik empieza a enrutar el dominio configurado hacia ese contenedor.

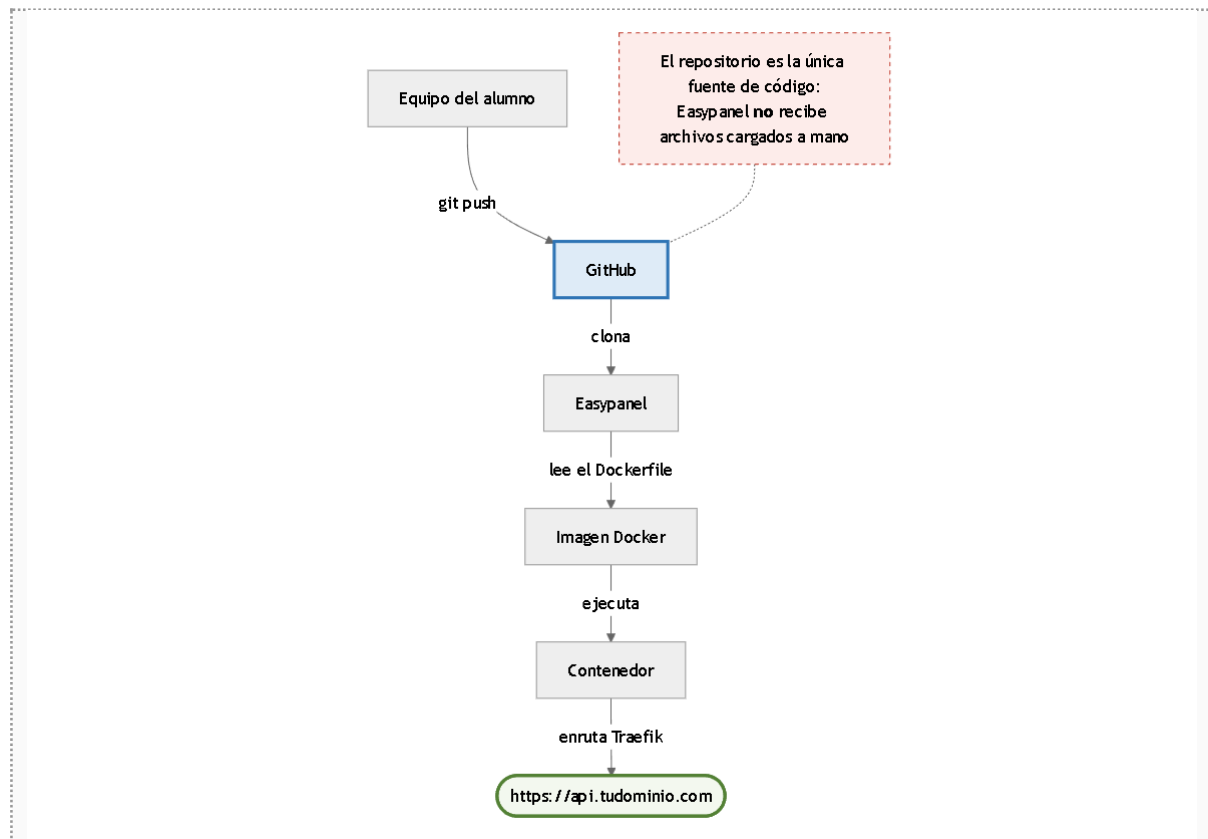


Figura 4.2. Flujo de construcción y publicación.

Obsérvese que el paso 1 no es un detalle de comodidad sino una decisión de arquitectura: **el repositorio es la única fuente de verdad**. No existe un estado del servidor que alguien haya producido a mano y que no esté en el código. Si el servidor se pierde entero, se reconstruye clonando y desplegando. Es la misma idea de la sección 3.2 —el entorno es un artefacto versionado— llevada al despliegue completo.

DATO

Si Easypanel no encuentra un Dockerfile, intenta deducir el tipo de aplicación con Nixpacks o Buildpacks. Funciona sorprendentemente bien para proyectos estándar, pero es una caja negra: cuando falla, no hay archivo que leer para entender por qué. Este proyecto trae sus propios Dockerfile justamente para no depender de eso.

4.4. Organización de los repositorios

El proyecto está distribuido en **dos repositorios independientes**:

Repositorio	Contenido	Servicio en Easypanel
MatyAlts/calculadora-backend	La API en FastAPI	api
MatyAlts/calculadora-frontend	El HTML y nginx	web

Esta organización se denomina **polirepositorio**, por oposición al monorepositorio, en el que ambas aplicaciones conviven en un único repositorio bajo subdirectorios. La elección entre una y otra es una de las discusiones recurrentes de la ingeniería de software y no tiene respuesta universal: depende de si los componentes se despliegan juntos o por separado, de si los mantienen los mismos equipos y de qué tan acoplados están sus cambios.

	Dos repositorios	Un repositorio
Build path en Easypanel	/ en cada servicio	/backend y /frontend
Ciclo de vida	Independiente por aplicación	Compartido
Permisos	Se otorgan por separado	Todo o nada
Integración continua	Un flujo por aplicación	Uno solo, con filtros por ruta
Clonado por el alumno	Dos comandos	Uno

PARA ENTENDER

Se eligieron dos repositorios porque es lo que hace coherente al proyecto entero. El `README.md` viene diciendo desde el principio que **son dos programas independientes**, que no comparten nada salvo el formato del JSON. Meterlos en el mismo repositorio contradiría ese mensaje. Además, es lo más habitual en la industria cuando el frontend y el backend los mantienen equipos distintos, que es justamente el caso de esta materia.

OJO ACÁ

Como cada repositorio contiene la aplicación **en su raíz**, el campo *Build path* de Easypanel va en /, no en /backend ni en /frontend. Si seguís una guía escrita para monorepositorio y ponés /backend, Docker va a buscar un directorio que en ese repositorio no existe, y la construcción falla con un mensaje que no menciona el Build path por ningún lado.

4.5. Creación del proyecto

Un **proyecto** en Easypanel es una agrupación lógica de servicios relacionados. Tiene además una consecuencia técnica que se vuelve central en la Clase 5: **los servicios de un mismo proyecto comparten una red interna** y pueden comunicarse entre sí sin exponer puertos.

1. Acceder a `https://easypanel.tudominio.com` (configurado en la Clase 2).
2. Crear un proyecto con el nombre `calculadora`.

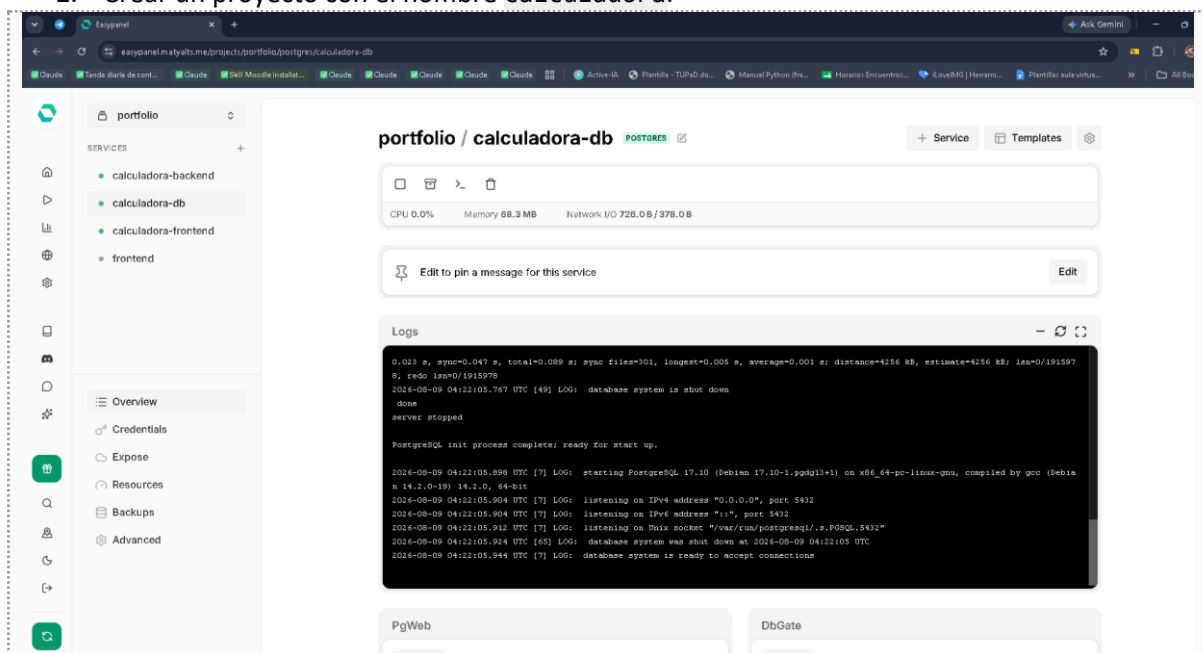


Figura 4.3. El proyecto con sus tres servicios en ejecución.

4.5.1. Dominio predeterminado

En **Settings** → **Domain** puede configurarse el dominio propio como dominio predeterminado del panel. A partir de ese momento, todo servicio que no tenga un dominio explícito recibe uno autogenerado bajo el dominio propio, en lugar de un dominio mágico de `easypanel1.host`.

DATO

Este ajuste, combinado con el registro comodín de la Clase 1, elimina de raíz el problema de resolución descrito en la sección 1.9. Los dominios autogenerados pasan a ser subdominios del dominio propio, que ningún proveedor filtra.

4.6. Conexión con GitHub

La primera vez que se configura un servicio con origen GitHub, Easypanel solicita autorización mediante la instalación de una aplicación de GitHub.

OJO ACÁ

Cuando GitHub te pregunte a qué repositorios le das acceso, elegí **solo los dos del proyecto**, no "All repositories". Es la aplicación directa del principio de mínimo privilegio de la sección 2.12.1: si mañana el servidor queda comprometido, el atacante llega hasta donde vos le dijiste que llegara. Darle acceso a todos tus repositorios para ahorrar dos clics es exactamente el tipo de decisión que se lamenta después.

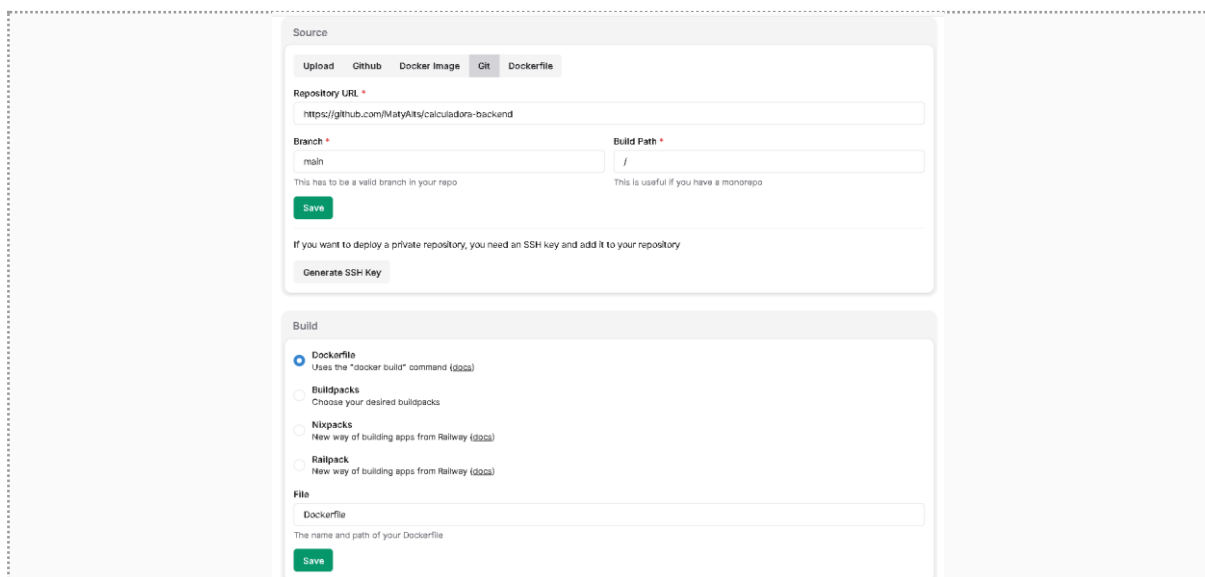
4.7. Despliegue del backend

En el proyecto calculadora: + **Service** → **App**, con el nombre `api`.

4.7.1. Source

Campo	Valor
Origen	GitHub
Repositorio	MatyAlts/calculadora-backend
Branch	main
Build path	/

Al existir un `Dockerfile` en la raíz del repositorio, Easypanel lo detecta y lo utiliza sin configuración adicional. El *Build path* es literalmente el contexto de construcción de la sección 3.4.1: el directorio que se envía al motor de Docker.



4.7.2. Environment

ORIGENES_PERMITIDOS=https://calculadora.tudominio.com

Este valor se completa ahora aunque el frontend todavía no esté desplegado, porque el dominio ya está decidido. Se verifica en la sección 4.10.

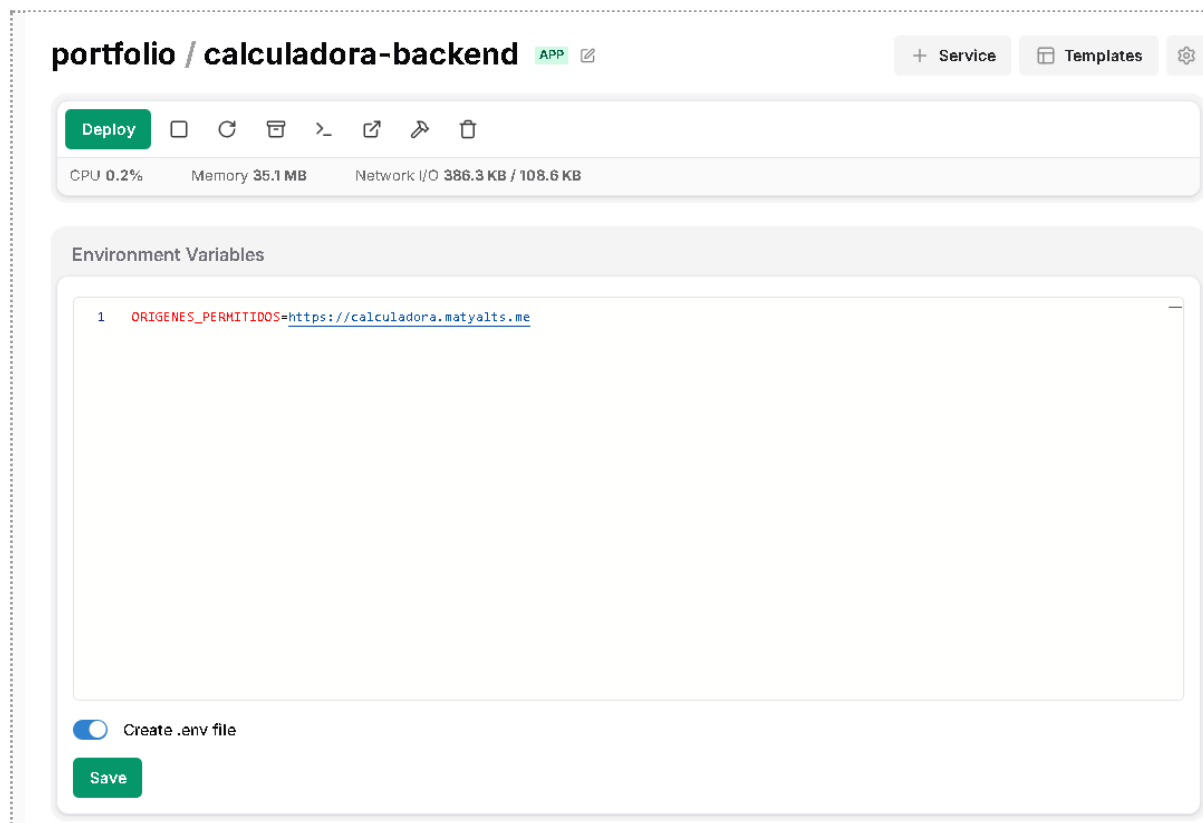


Figura 4.5. El origen autorizado se declara como variable de entorno, no en el código.

4.7.3. Domains & Proxy

Campo	Valor	Origen del valor
Dominio	api.tudominio.com	Resuelve por el registro comodín de la Clase 1
Puerto	8000	El que declara el Dockerfile y usa uvicorn
HTTPS	Activado	Emisión automática por Let's Encrypt

⚠ OJO ACÁ

El campo Puerto es el **puerto interno del contenedor**, no un puerto del servidor. Le está diciendo a Traefik: *cuando llegue algo para api.tudominio.com, mandáselo a este contenedor, al 8000*. Nadie está abriendo el 8000 en el VPS. De hecho, si corrés nmap desde afuera, el 8000 sigue cerrado. Traefik entra por el 443 y reparte por adentro.

4.7.4. Despliegue y verificación

Iniciar el despliegue y observar la pestaña **Logs**. La primera construcción demora varios minutos: descarga la imagen base de Python y todas las dependencias. Las siguientes son mucho más rápidas, por el caché de capas visto en la sección 3.4.3.

Verificaciones:

URL	Resultado esperado
https://api.tudominio.com/api/salud	{"estado": "ok"}
https://api.tudominio.com/docs	Documentación interactiva de FastAPI

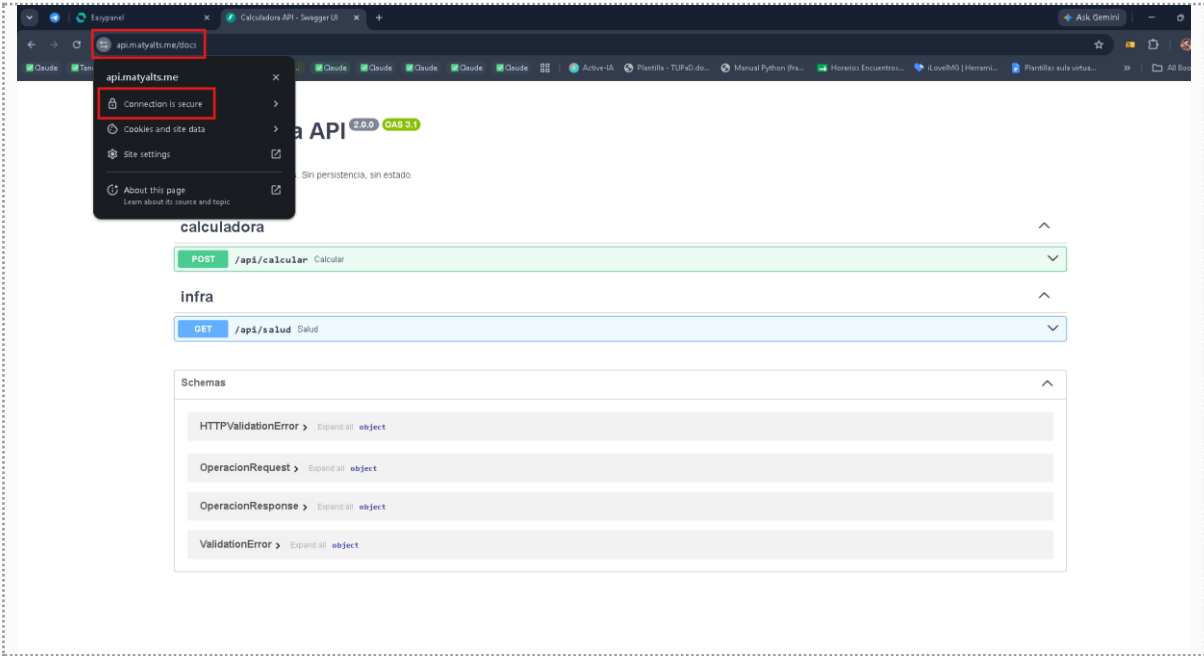


Figura 4.6. Documentación interactiva de la API, publicada con certificado válido.

 **EXPERIMENTO**

Abri `https://api.tudominio.com/docs` y hacé una operación desde ahí, sin usar el frontend. Funciona perfecto. Y **no hay CORS de por medio**, porque la página está servida por el mismo origen que la API.

Esa observación es media clase de CORS: el problema no es la API, es la combinación de dos orígenes distintos. Guardalo, porque en diez minutos van a ver el mismo endpoint fallando desde el frontend.

4.8. Anatomía del enrutamiento: cómo decide Traefik

4.8.1. El modelo: punto de entrada, enrutador y servicio

Traefik organiza su trabajo en tres conceptos encadenados, y conocerlos convierte sus mensajes de error en información útil.

Concepto	Qué es	En este proyecto
Punto de entrada (<i>entrypoint</i>)	Un puerto en el que Traefik escucha	El 80 y el 443
Enrutador (<i>router</i>)	Una regla que decide qué peticiones toma	Host(<code>api.tudominio.com</code>)
Middleware	Una transformación intermedia opcional	La redirección de HTTP a HTTPS
Servicio (<i>service</i>)	El destino interno al que se entrega	El contenedor <code>api</code> , puerto 8000

La secuencia es: llega una petición a un punto de entrada, se evalúan los enrutadores hasta encontrar uno cuya regla coincida, se aplican sus middlewares, y se entrega al servicio asociado. Esa cadena explica de forma exacta los dos códigos de error más frecuentes del capítulo:

- **404:** ningún enrutador coincidió. El nombre solicitado no está asociado a ningún servicio. Es un problema de configuración de dominio.
- **502:** el enrutador coincidió y el servicio **no contestó** o contestó algo inválido. Traefik encontró a quién entregarle la petición, fue, y no había nadie. Es un problema de puerto o de interfaz de escucha.

Que sean dos errores distintos no es casual: cada uno señala una etapa distinta de la cadena. Confundirlos manda a buscar el problema al lugar equivocado.

4.8.2. Alojamiento virtual en funcionamiento

Los dos servicios del proyecto se ejecutan en el mismo servidor y comparten una única dirección IP pública. La distinción la realiza Traefik leyendo la cabecera `Host` de cada petición, mecanismo introducido en la sección 1.3 y fundamentado en la sección 4.2.

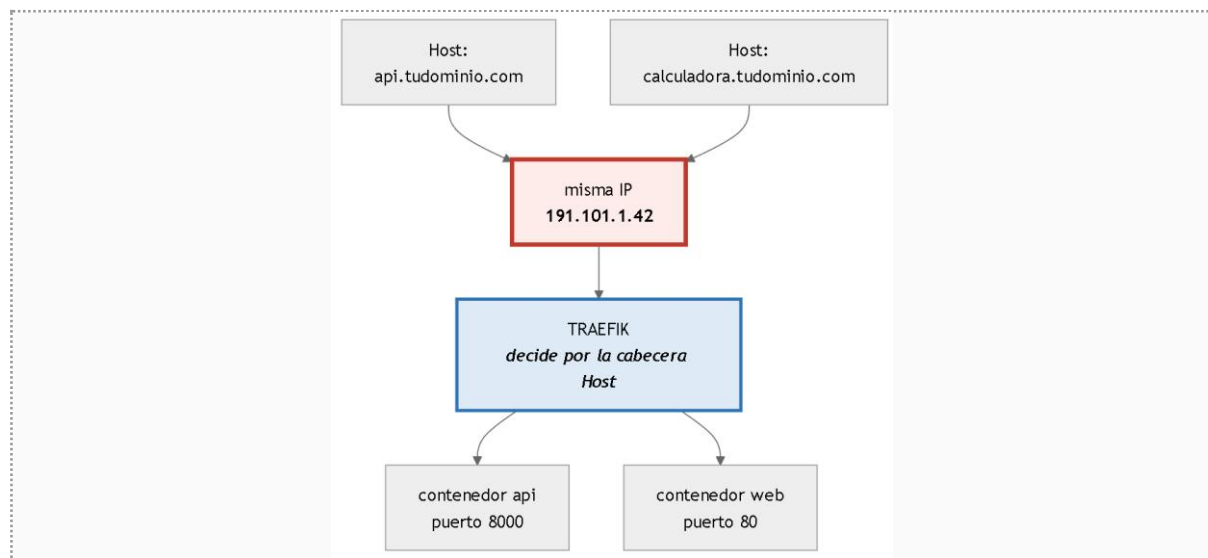


Figura 4.7. Alojamiento virtual. Un único servidor y una única dirección IP atienden dos dominios distintos; la cabecera `Host` determina el destino.

Vale precisar que en una conexión HTTPS el nombre viaja **dos veces**: primero en claro, en la extensión SNI del saludo TLS, para que Traefik sepa qué certificado presentar; y después, cifrado, en la cabecera `Host` de la petición. Los dos valores deberían coincidir, y un desacuerdo entre ambos es una anomalía que los proxies modernos rechazan.

4.8.3. Emisión del certificado: el protocolo ACME

Antes de 2015, obtener un certificado era un trámite manual, pago y anual, y esa fricción era la razón principal de que la mayor parte de la web viajara sin cifrar. Let's Encrypt cambió esa situación con dos decisiones combinadas: emitir certificados gratuitos y, sobre todo, **automatizar completamente la emisión mediante un protocolo**. Ese protocolo se llama **ACME** (*Automatic Certificate Management Environment*) y está normado en la RFC 8555.

Lo que ACME resuelve es un problema concreto: cómo una autoridad certificadora puede comprobar, sin intervención humana, que quien pide un certificado para `api.tudominio.com` controla efectivamente ese nombre. La respuesta es un **desafío**: la autoridad pide una demostración que solo puede dar quien controla el dominio.

Desafío	Qué pide demostrar	Cómo
HTTP-01	Control del servidor al que apunta el nombre	Servir un archivo con un valor dado, por el puerto 80
DNS-01	Control de la zona DNS	Publicar un registro TXT con un valor dado
TLS-ALPN-01	Control del servicio TLS	Responder el saludo TLS de una forma particular

Al activar HTTPS, Traefik utiliza el desafío **HTTP-01**:

1. Traefik solicita un certificado para `api.tudominio.com`.
2. Let's Encrypt responde con un valor a publicar y **se conecta al dominio por el puerto 80** para verificarlo.
3. Si el valor coincide, emite el certificado.

De este procedimiento se desprenden los dos requisitos que vienen arrastrándose desde las clases anteriores, y ahora se entiende exactamente por qué:

Requisito	Establecido en	Por qué
El dominio debe resolver hacia el VPS	Clase 1	Let's Encrypt resuelve el nombre para saber a dónde conectarse
El puerto 80 debe estar abierto	Clase 2	El desafío HTTP-01 se valida por ese puerto, no por el 443

El desafío DNS-01 de la tercera fila es el que se mencionó en la sección 1.12.2 al explicar por qué el certificado comodín es otra cosa: como no requiere conectarse al servidor sino publicar un registro en la zona, es el único que puede validar un nombre con asterisco, y por eso exige credenciales de API del proveedor DNS.

DATO

Los certificados de Let's Encrypt duran **90 días**, y la renovación se intenta cuando faltan 30. Esa duración corta es deliberada: obliga a que la renovación esté automatizada, porque a mano sería insostenible. Un certificado que dura tres meses y se renueva solo es más seguro que uno que dura tres años y que nadie revisa. Consecuencia práctica: el puerto 80 tiene que quedar abierto **para siempre**, no solo el día del despliegue.

OJO ACÁ

Si el certificado no sale, el 90 % de las veces es una de esas dos cosas, y ninguna se arregla desde Easypanel. Antes de tocar un solo botón del panel: `nslookup` del dominio y `nmap` del puerto 80. Si esos dos dan bien, entonces sí, mirá los logs de Traefik.

4.9. Despliegue del frontend

+ Service → App, con el nombre web.

4.9.1. Source

Campo	Valor
Repositorio	MatyAlts/calculadora-frontend
Branch	main
Build path	/

4.9.2. Environment

`API_URL=https://api.tudominio.com`

Esta es la variable que lee `docker-entrypoint.sh` al arrancar el contenedor para generar `config.js`, tal como se vio en la sección 3.7.2.

Tres detalles sobre este valor, y los tres rompen el despliegue:

- **Con https://.** Si ponés `http://`, el navegador bloquea el pedido por contenido mixto: una página segura no puede consumir un recurso inseguro.
- **Sin barra al final.** El JavaScript le concatena `/api/calcular`. Con barra queda una doble barra.
- **Sin la ruta del endpoint.** Va el dominio pelado, no `https://api.tudominio.com/api/calcular`.

Campo	Valor
Dominio	calculadora.tudominio.com
Puerto	80
HTTPS	Activado

El puerto acá es 80, no 8000. nginx escucha en el 80. Este es el error más repetido de la clase: se copia la configuración del backend y se cambia solo el dominio. El síntoma es un **502 Bad Gateway**, que —según el modelo de la sección 4.8.1— significa que el enrutador sí coincidió pero el servicio no contestó en el puerto que le dijeron.

En la pestaña **Logs** del servicio web debe figurar:

Si en cambio aparece el aviso de que la variable no está definida, falta cargarla en Environment.

Con ambos servicios en funcionamiento, la aplicación todavía no funciona. Falta autorizar al frontend a consumir la API. Esta sección explica por qué hace falta esa autorización, qué la impone y —lo más importante— qué protege y qué no.

Un **origen** es la tripleta formada por el **esquema**, el **anfitrión** y el **puerto** de una URL. La definición está normada en la RFC 6454 y es estricta: dos URL pertenecen al mismo origen si y solo si coinciden los tres componentes, comparados literalmente.

Nótese que la ruta **no forma parte del origen** y que la comparación no interpreta nada: `tudominio.com` y `www.tudominio.com` son dos anfitriones distintos, aunque para una persona sean "el mismo sitio".

La regla de fondo la introdujo Netscape en 1995 y hoy es el pilar de la seguridad del navegador: **el código de un origen no puede leer los datos de otro origen**. La razón se entiende con un ejemplo. Si estás autenticado en tu banco y abris otra pestaña con un sitio cualquiera, ese sitio podría hacer una petición al banco; el navegador, que guarda las cookies por dominio, las adjuntaría

automáticamente, y el banco respondería con tus datos. Sin la política de mismo origen, ese sitio podría **leer** la respuesta. Con ella, la petición puede llegar a emitirse pero el navegador no le entrega la respuesta al código que la pidió.

Esa política es deliberadamente restrictiva, y por eso hizo falta un mecanismo para relajarla de forma controlada cuando el acceso entre orígenes es legítimo. Ese mecanismo es **CORS** (*Cross-Origin Resource Sharing*), y su lógica es que **el permiso lo otorga el servidor de destino**, no el que pide. La API declara, mediante cabeceras de respuesta, qué orígenes están autorizados a leerla; el navegador lee esa declaración y decide.

4.10.3. La verificación previa

Para las peticiones que pueden tener efectos —las que no son una simple lectura— el navegador antepone una **verificación previa** (*preflight*): una petición con el método **OPTIONS** que pregunta al servidor si va a autorizar la petición real. Si la respuesta no autoriza explícitamente el origen, el método y las cabeceras solicitadas, **la petición real nunca se emite**.

El disparador de esa verificación merece precisarse, porque explica por qué algunas peticiones la tienen y otras no. Se consideran *simples* —y viajan sin verificación previa— las peticiones **GET**, **HEAD** y **POST** cuyo tipo de contenido sea uno de los tres históricos de los formularios HTML. Cualquier otra cosa dispara la verificación, y **Content-Type: application/json es justamente "cualquier otra cosa"**. Por eso la calculadora, que envía JSON, siempre produce dos peticiones donde parecía haber una.

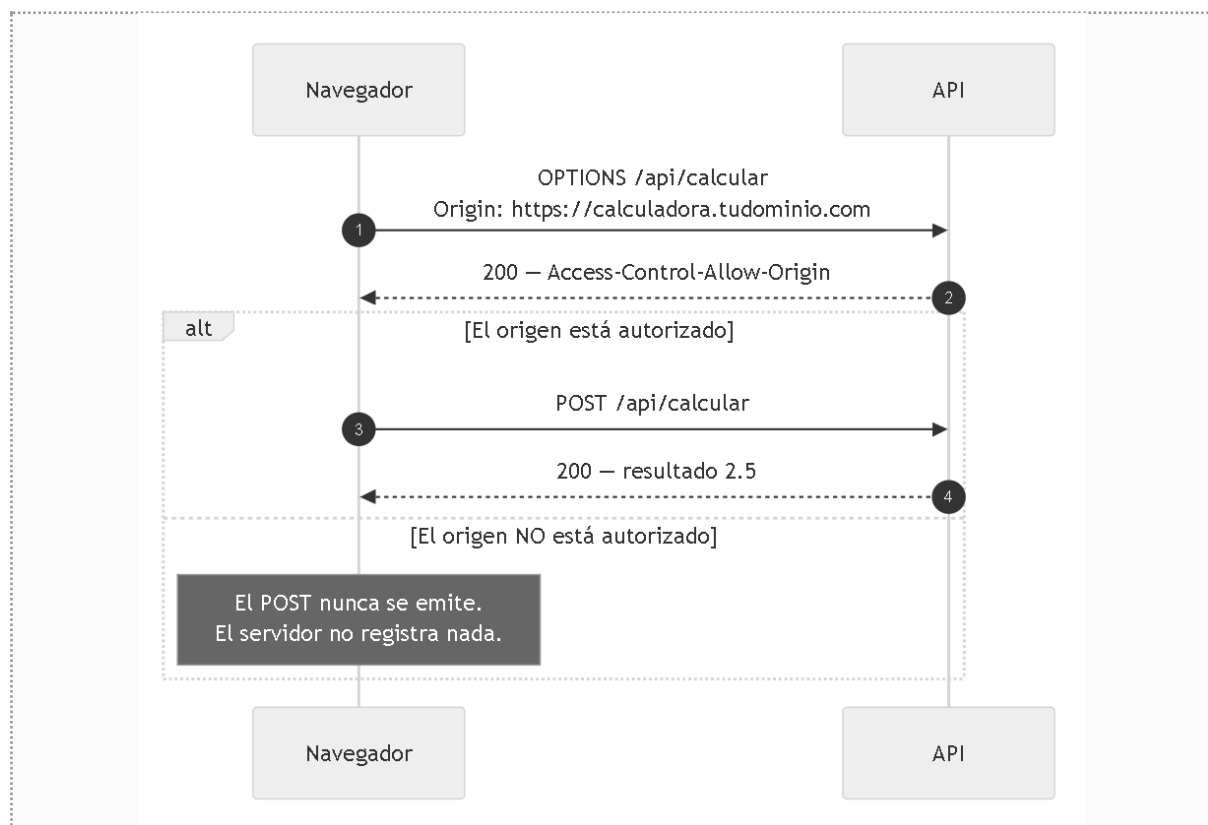


Figura 4.8. Verificación previa y petición efectiva. Si el preflight no autoriza el origen, la petición real no llega a emitirse.

Se verifica que la variable del servicio `api` contenga exactamente el dominio del frontend y se redespiega el servicio para que tome el cambio:

```
ORIGENES_PERMITIDOS=https://calculadora.tudominio.com
```

4.10.4. Qué cuenta como origen distinto

Se consideran orígenes distintos	Consecuencia
http:// frente a https://	Bloqueado, aunque el dominio coincida
tudominio.com frente a www.tudominio.com	Bloqueado; deben declararse ambos, separados por comas
Con barra final: https://tudominio.com/	No coincide. El origen se declara sin barra
Puertos distintos	Orígenes distintos

Las cuatro filas son consecuencias directas de la definición literal de la sección 4.10.1. No hay excepciones ni tolerancia: la comparación es de cadenas.

4.10.5. Qué NO protege CORS

Este es el punto que más se malinterpreta, y conviene enunciarlo sin rodeos: **CORS no protege la API**. Es una política que el navegador aplica **a favor del usuario** para impedir que un sitio cualquiera lea datos de otro en su nombre. No es un control de acceso del servidor.

Un cliente que no sea un navegador —`curl`, un script de Python, otro servidor— **ignora CORS por completo**, porque no tiene usuario a quien proteger ni cookies ajenas que adjuntar. Si la API tiene que restringir quién la consume, eso se hace con autenticación y autorización en el servidor, que es un mecanismo completamente distinto.

⚠ OJO ACÁ

Acordate de lo más importante de CORS, que está explicado en el `README.md` del proyecto: **lo aplica el navegador, no el servidor**. El log del backend te va a mostrar el pedido entrando y saliendo con un 200 impecable, mientras el frontend dice que falló. Los dos tienen razón. El servidor contestó; el navegador tiró la respuesta a la basura porque no traía el permiso.

Si el fetch falla pero un `curl` a la misma URL anda, ya sabés dónde mirar. Y no es en el servidor.

4.11. Verificación funcional

Abrir `https://calculadora.tudominio.com` con las herramientas de desarrollo del navegador activas (F12) y ejecutar una operación.

4.11.1. Pestaña Red

Deben registrarse **dos** peticiones hacia el dominio de la API:

Método	Recurso	Código
OPTIONS	/api/calcular	200
POST	/api/calcular	200

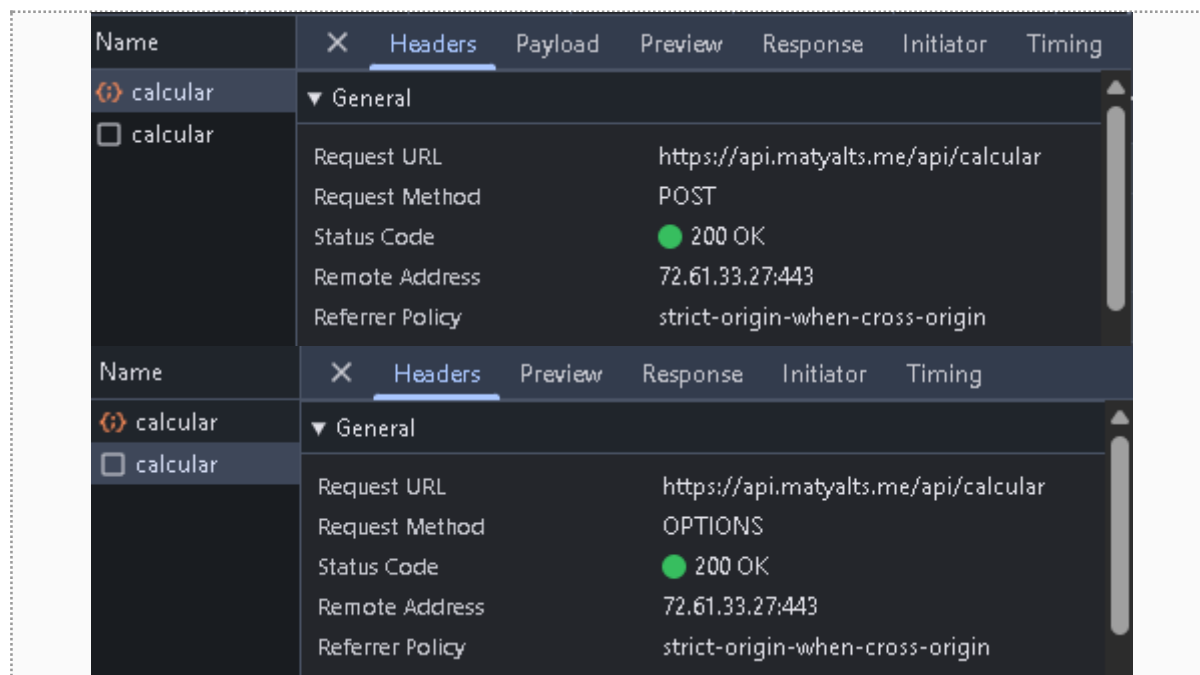


Figura 4.9. El preflight y la petición efectiva, ambas correctas.

Que aparezcan exactamente dos, y en ese orden, es la confirmación empírica de todo lo explicado en la sección 4.10.3. Si solo aparece una, o el `OPTIONS` no da 200, el problema está en la autorización de origen y no en la lógica de la aplicación.

4.11.2. Pestaña Consola

Sin mensajes de error.

4.11.3. Casos límite

Entrada	Resultado esperado	Código HTTP
$10 \div 0$	Mensaje de división por cero	400
$1e308 \times 10$	Mensaje de resultado fuera de rango	400
Botón Limpiar	Ambos campos en 0	—
Certificado del sitio	Válido, emitido por Let's Encrypt	—

EXPERIMENTO

Con la aplicación andando, hacé este pedido desde la terminal:

```
curl -X POST https://api.tudominio.com/api/calcular -H "Content-Type: application/json" -H "Origin: https://un-origen-prohibido.com" -d '{"a":1,"b":1,"operacion":"suma"}'
```

Devuelve 2.0. Con un origen prohibido. Sin error. Porque **curl no es un navegador y no tiene que proteger a nadie**. Es la demostración definitiva de lo que dice la sección 4.10.5: CORS es una política del cliente, no una defensa del servidor. Si alguien creía que CORS protegía la API, acá se le termina de caer la idea.

4.12. Operación

Acción	Dónde	Cuándo se usa
Ver el registro en vivo	Pestaña Logs del servicio	Diagnóstico permanente
Redesplegar	Botón Deploy	Tras cambiar variables de entorno

Acción	Dónde	Cuándo se usa
Ver despliegues anteriores	Pestaña Deployments	Para volver a una versión previa
Reiniciar el contenedor	Botón Restart	Cuando el proceso queda colgado

⚠ OJO ACÁ

Cambiar una variable de entorno no tiene efecto hasta que redespliegues. Las variables se leen al arrancar el proceso. Vas a corregir `ORIGENES_PERMITIDOS`, vas a guardar, vas a probar, va a seguir fallando, y vas a pensar que pusiste mal el valor. No: te faltó el redeploy.

4.13. Seguridad y evolución del transporte web

El despliegue de este capítulo funciona y es correcto. Esta sección enumera qué le falta para parecerse a un despliegue profesional y hacia dónde evoluciona la capa sobre la que está construido.

4.13.1. Cabeceras de seguridad

Un proxy inverso es el lugar natural para agregar cabeceras de respuesta que endurecen el comportamiento del navegador. Las dos más importantes:

HSTS (*HTTP Strict Transport Security*, RFC 6797) le indica al navegador que ese dominio debe visitarse **siempre** por HTTPS, durante un plazo declarado. La diferencia con una simple redirección de HTTP a HTTPS es sutil y significativa: la redirección todavía implica una primera petición insegura que alguien podría interceptar; con HSTS activo, el navegador ni siquiera la emite. Es el mecanismo detrás de la lista de precarga que se mencionó en la sección 1.13 a propósito de los dominios `.app` y `.dev`.

CSP (*Content Security Policy*) declara de qué orígenes puede el navegador cargar scripts, estilos e imágenes. Es la defensa estándar contra la inyección de código en el navegador, y su lógica es la misma de todo este capítulo: una lista blanca declarada por el servidor y aplicada por el cliente.

4.13.2. HTTP/2 y HTTP/3

El HTTP/1.1 de 1997 que fundamenta este capítulo tiene un problema estructural: una conexión transporta una petición por vez, y las siguientes esperan. HTTP/2 (2015) lo resolvió multiplexando muchas peticiones sobre una única conexión TCP, y HTTP/3 (2022) fue más lejos reemplazando TCP por QUIC, un protocolo sobre UDP que elimina el bloqueo de cabeza de línea y sobrevive a los cambios de red del cliente.

Para este práctico el cambio es transparente —Traefik negocia la versión más alta que el navegador soporte, sin configuración— pero importa saber que **la cabecera Host y el modelo de orígenes no cambiaron**. Lo que evolucionó es el transporte; la semántica de HTTP, normada hoy en la RFC 9110, es la misma. Es un buen ejemplo de una separación de capas bien hecha.

📌 DATO

Que HTTP/3 corra sobre UDP tiene una consecuencia operativa que sorprende a quien viene de la Clase 2: un firewall que solo permite TCP en el 443 **no bloquea HTTP/3, lo degrada**. El navegador intenta QUIC, no recibe respuesta, y vuelve silenciosamente a HTTP/2 sobre TCP. La página carga igual, un poco más lento, y nada en ningún log dice qué pasó.

4.14. Errores frecuentes

Se recomienda diagnosticar a partir del mensaje concreto de la consola del navegador y no del comportamiento aparente de la aplicación.

Síntoma	Causa	Resolución
CORS policy en la consola	El dominio del frontend no figura en ORIGENES_PERMITIDOS	Revisar esquema, www y barra final (sección 4.10.4); redespargar
Mixed Content	El frontend va por HTTPS y API_URL está en HTTP	Corregir la variable a https://
ERR_CONNECTION_REFUSED hacia 127.0.0.1	config.js no se generó	Verificar API_URL en el servicio web
502 Bad Gateway	El enrutador coincidió pero el servicio no contestó	Revisar el puerto: backend 8000, frontend 80
502 con el puerto correcto	El proceso escucha en 127.0.0.1	Verificar --host 0.0.0.0 en el CMD (sección 3.5.5)
404 desde Traefik	Ningún enrutador coincidió: el dominio no está asociado a ningún servicio	Revisar Domains & Proxy
El certificado no se emite	El DNS no resuelve, o el puerto 80 está cerrado	nslookup y nmap antes de tocar el panel
El certificado dejó de renovarse a los tres meses	Se cerró el puerto 80 después del despliegue	Reabrirlo: la renovación lo necesita siempre (sección 4.8.3)
La construcción falla	Build path incorrecto	Debe ser / en ambos repositorios
El cambio de variable no surte efecto	Falta redespargar	Botón Deploy

PARA ENTENDER

Fijate que casi todos los síntomas de esta tabla apuntan a un lugar equivocado. Un 502 parece un problema del servidor y es un número mal puesto en un formulario. Un error de CORS parece un problema de red y es una cadena de texto. Por eso el diagnóstico arranca siempre por el **mensaje exacto**, nunca por la intuición.

4.15. Actividades

Actividad 1 — Ruptura y reparación de CORS.

Modificar ORIGENES_PERMITIDOS con un dominio incorrecto, redespargar, documentar el mensaje exacto de la consola y repararlo. Indicar en qué etapa de la secuencia de la sección 4.10.3 se detuvo la operación y por qué la petición POST no llegó a emitirse.

Actividad 2 — Reproducción del 502.

Cambiar el puerto del servicio web de 80 a 8000, observar el error y explicar, en términos del modelo de punto de entrada, enrutador y servicio de la sección 4.8.1, por qué Traefik responde exactamente ese código y no un 404.

Actividad 3 — Demostración del alojamiento virtual.

Verificar con nslookup que ambos dominios resuelven a la misma dirección IP y explicar cómo el servidor distingue a cuál de los dos servicios corresponde cada petición. Comprobar además, con curl -v, que el nombre viaja tanto en el saludo TLS como en la cabecera Host.

Actividad 4 — Análisis de la verificación previa.

Capturar la petición OPTIONS en la pestaña Red y documentar las cabeceras Access-Control-Allow-Origin, Access-Control-Allow-Methods y Access-Control-Max-Age de la respuesta. Explicar qué efecto tiene la última sobre la cantidad de peticiones observadas al repetir la operación varias veces seguidas.

Actividad 5 — Publicación de un tercer subdominio.

Publicar la documentación de la API en docs.tudominio.com sin tocar el panel DNS, demostrando el funcionamiento del registro comodín.

Actividad 6 — El desafío HTTP-01, observado.

Publicar un servicio nuevo con HTTPS y, mientras se emite el certificado, seguir el registro de Traefik. Identificar el momento en que Let's Encrypt se conecta por el puerto 80 y qué ruta solicita. Explicar por qué esa validación no podría hacerse por el 443.

Actividad 7 — CORS no es un control de acceso. (analítica)

A partir del experimento de la sección 4.11, redactar en un párrafo la respuesta a esta pregunta: si un competidor quisiera consumir tu API desde su propio servidor, ¿lo impediría `ORIGENES_PERMITIDOS`? Justificar y proponer qué mecanismo sí lo impediría.

4.16. Síntesis

1. El alojamiento virtual existe porque HTTP/1.1 hizo **obligatoria la cabecera Host**, y funciona con HTTPS gracias a **SNI**. Sin esas dos piezas, cada sitio necesitaría su propia dirección IP.
2. Un **proxy inverso** ocupa los puertos 80 y 443, termina el cifrado y reparte por nombre. Traefik además **descubre** los servicios en lugar de leerlos de un archivo.
3. Easypanel coordina **Docker, Traefik y Let's Encrypt**. Diagnosticar consiste en determinar cuál de los tres falló.
4. El código se publica **desde un repositorio**, nunca cargando archivos: el repositorio es la única fuente de verdad.
5. Con repositorios separados, el **Build path es /**.
6. El **puerto de Domains & Proxy es el interno del contenedor**, y no implica abrir nada en el servidor.
7. En el modelo de Traefik, **404 significa que ningún enrutador coincidió y 502 que el servicio no contestó**. Son dos etapas distintas de la cadena.
8. El certificado se emite por **ACME**, con el desafío HTTP-01: requiere **DNS resuelto y puerto 80 abierto**, y lo requiere también en cada renovación.
9. Un **origen** es esquema, anfitrión y puerto, comparados literalmente. **CORS lo aplica el navegador y no protege la API**: protege al usuario de que un sitio ajeno lea datos en su nombre.
10. Una variable de entorno modificada **no tiene efecto hasta el redesplicue**.

4.17. Referencias y lecturas complementarias

La normativa de HTTP fue reorganizada en 2022 y conviene citar las versiones vigentes: la **RFC 9110** define la semántica del protocolo —métodos, códigos de estado y cabeceras, incluida **Host**—, la **RFC 9112** especifica HTTP/1.1, la **RFC 9113** HTTP/2 y la **RFC 9114** HTTP/3, que corre sobre el transporte QUIC de la **RFC 9000**. El concepto de origen está normado en la **RFC 6454** (*The Web Origin Concept*), y la especificación normativa actual de CORS no es una RFC sino el *Fetch Standard* del WHATWG, disponible en fetch.spec.whatwg.org.

Para la capa de seguridad del transporte: la **RFC 8446** define TLS 1.3, la **RFC 6066** incorpora la extensión SNI, la **RFC 6797** define HSTS y la **RFC 8555** especifica el protocolo ACME de emisión automática de certificados. La documentación operativa de Traefik, en doc.traefik.io, y la de Let's Encrypt, en letsencrypt.org/docs, son las referencias de consulta cotidiana; esta última explica los tres desafíos con más detalle que la RFC.

Como bibliografía de estudio, I. Grigorik, *High Performance Browser Networking* (O'Reilly, 2013, disponible gratuitamente en hpbk.co) cubre TLS, HTTP/2 y el comportamiento real del navegador con un nivel de detalle poco frecuente. Para la política de mismo origen y el modelo de seguridad del navegador, M. Zalewski, *The Tangled Web* (No Starch Press, 2011) sigue siendo la explicación más completa de por qué la web es insegura por construcción y qué mecanismos la sostienen. La documentación de MDN sobre CORS es la mejor referencia práctica de consulta rápida.